



Talent Assessment [Senior Frontend Engineer]

Objective

This assessment evaluates your ability to convert production-ready UI designs into a responsive web application while demonstrating clean architecture, component-based development, API integration, and backend implementation.

You will find the Figma design containing 3–5 website sections below. Your task is to implement the design using ReactJS for the frontend and either Node.js (Express) or Django for the backend within the allocated time.

Main Figma File:

<https://www.figma.com/design/uraGfsYkol0bQjarnnhfs/Technical-Assesment--SFE-?node-id=0-1&m=dev&t=1XhIPqgsFQ7yMsWY-1>

Prototype:

<https://www.figma.com/proto/uraGfsYkol0bQjarnnhfs/Technical-Assesment--SFE-?node-id=0-1&t=1XhIPqgsFQ7yMsWY-1>

Project : Figma to Responsive Web Application

Build the provided design as accurately as possible while following modern frontend development best practices

1. Project Setup (30 minutes):

- A. Initialize a ReactJS project using your preferred build tool (Vite or Create React App).
- B. Set up a clean and scalable folder structure.
- C. Configure routing if applicable.
- D. Create a backend using either: (1) Node.js (Express) Or, (2) Django

E. Create an API endpoint to serve the website data/content.

2. UI Development (120 minutes)

- A. Implement all sections from the provided Figma design.
- B. Develop reusable React components.
- C. Match typography, spacing, colors, icons, and layout as closely as possible.
- D. Ensure semantic HTML structure.
- E. Use proper component hierarchy and state management where necessary.

3. Backend Integration (90 minutes)

Create a backend service that exposes the required data via REST APIs.

Examples:

GET /api/home

GET /api/features

GET /api/testimonials

Requirements:

- Consume backend APIs from React
- Display dynamic content
- Handle loading states
- Handle API errors gracefully

The backend does not need authentication or database integration. Static JSON responses are acceptable.

4. Responsive Design & Polish (30 minutes)

- A. Ensure mobile-first approach
- B. Implement proper error handling
- C. Implement Cross-browser compatibility
- D. Optimize performance

Technology Requirements

- **Frontend:** ReactJS, JavaScript or TypeScript
- **Backend:** Node.js (Express) OR, Django
- **Styling:** CSS, SCSS, Tailwind CSS, CSS Modules, Styled Components (Your preference)
- **State Management:** Choose either Redux or React Context
- **Version Control:** Git with regular commits
- **Testing:** Optional (React Testing Library)

API Requirement

Create REST APIs to provide all page content.

Example:

GET /api/home

Response

```
</> JSON

{
  "heroTitle": "...",
  "heroDescription": "...",
  "features": [],
  "services": [],
  "footer": {}
}
```

Using static JSON data is acceptable.

Evaluation Criteria

Your submission will be evaluated based on:

- **UI Accuracy:** Fidelity to the provided Figma design, Attention to spacing, alignment, typography, and visual consistency
- **Code Quality:** Clean, maintainable code with proper TypeScript typing
- **Functionality:** Fetching dynamic data to frontend as specified
- **Responsive Design:** Desktop (Widescreen) and Mobile responsiveness
- **React Best Practices:** State management, Hooks usage, Component composition, Performance considerations

- **Backend Implementation:** API structure, Code organization, Integration with frontend
- **Problem-solving:** Overall engineering approach, Handling edge cases, Technical decisions

Submission Requirements (30 minutes)

Candidates must submit the following **via email** to hr@d4t4crunch.com:

- **Documentation:** Create a comprehensive README.md that includes:
 - Setup instructions
 - Project structure
 - Technologies used
 - Assumptions made
 - Future improvements
- **Code Repository:**
 - Push your code to a GitHub repository
 - Ensure the repository is public or accessible to reviewers
 - Maintain a clean commit history
- **Optional Bonus Points:**
 - Deployed application (Vercel, Netlify, etc.)
 - Unit tests
 - Animations and transitions
 - TypeScript implementation
 - Performance optimization
 - Basic SEO best practices

Assessment Tips

- Spend the first 10-20 minutes **understanding the Figma design**.
- **Build reusable components** instead of duplicating code.
- **Prioritize responsiveness** from the beginning.
- Follow **pixel-perfect implementation** as closely as possible.
- Ensure the site is **fully functional** before polishing.
- Leave sufficient **time for documentation** and final review.

PLEASE NOTE: AI Tool Usage

We don't restrict the use of AI coding assistants for this assessment. What matters most is your understanding of **concepts, problem solving abilities, and implementation**

skills. Whether you use AI tools or not, be prepared to explain your architectural decisions and reasoning during the follow up discussion. Our focus is on evaluating your ability to deliver functional solutions, not how the code was generated.

DEADLINE: 4TH JULY 2026 @ 10:00 AM